Disaggregated TCU Integration on SimX

Chulhyung Park
Mentor: Shinnung Jeong

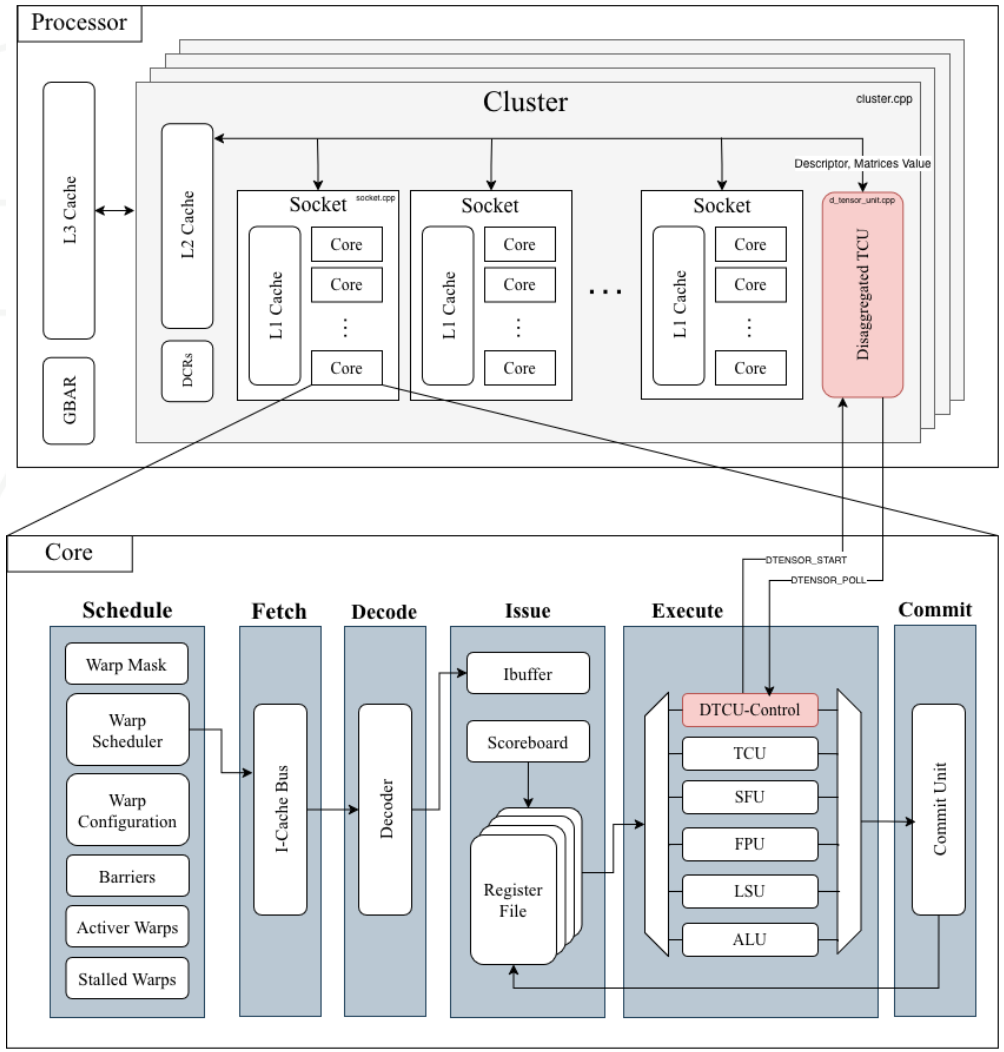
Georgia Tech

Problems

- Motivated by the paper from UC Berkeley
  - *Virgo: Cluster-level Matrix Unit Integration in GPUs for Scalability and Energy Efficiency*
- Current Vortex's TCU lives inside the SIMT core
  - Core decodes operands, executes matrix computation, writebacks results
  - Dependent on core's Register Files as they are used for operand/accumulator storage
- **Problems:**

  1) Limited operand/accumulator bandwidth due to high register pressure
  2) Limited scalability
- Virgo cluster-level disaggregated TCU
  - Offload matrix computation off from the pipeline
  - Bypass the core RF & use memory for data exchange
  - Enable larger-granularity operations and better scalability

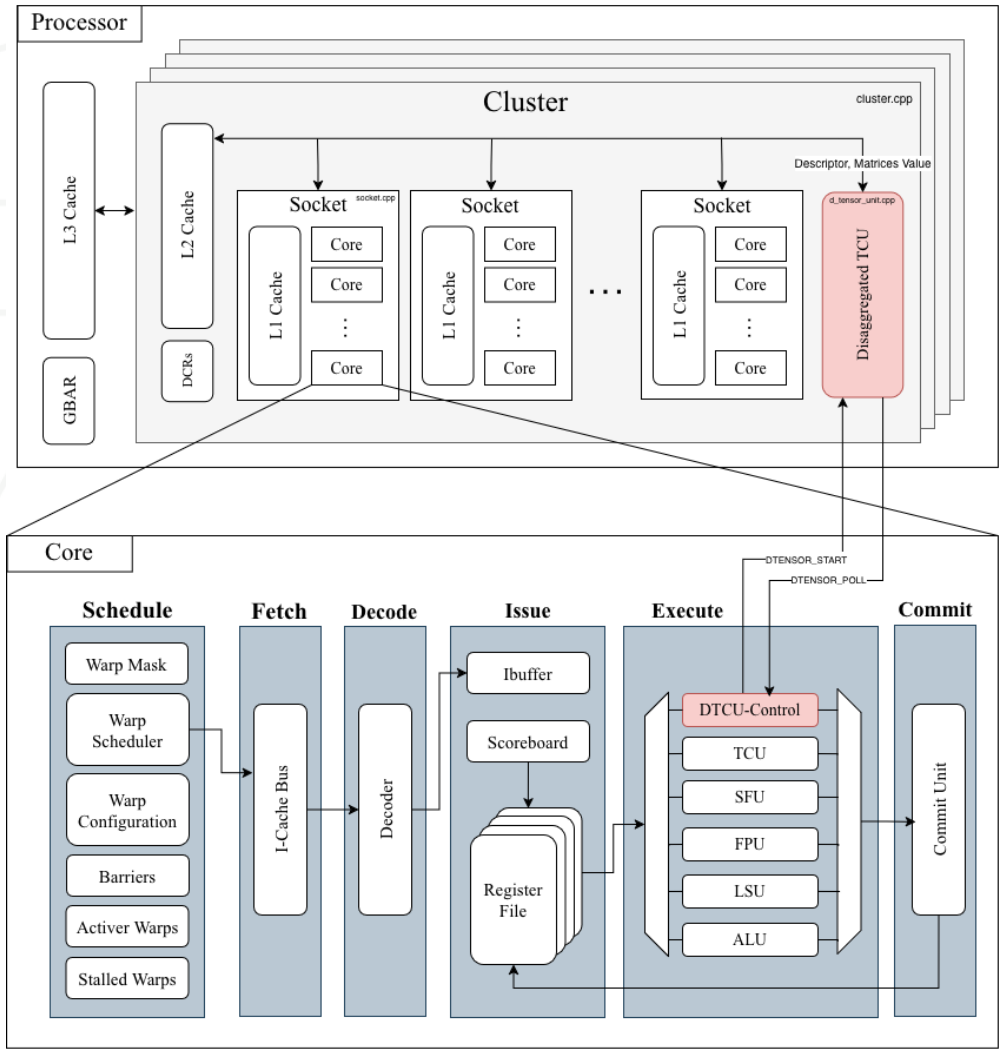
Abstract



- DTCU is a cluster-level module
- DTCU is controlled by a separate FU
  - DTCU_Control
- Custom instructions
  - *RISCV_CUSTOM0*, *func7 = 2*
  - DTENSOR_START (func3 = 1)
    - Passes descriptor pointer
  - DTENSOR_POLL (func3 = 2)
    - Core polls if completed
- Data exchange via memory through L2 Cache simulation
  - Descriptor holds pointers to operands, accumulator, output and metadata
  - Stores back to location specified in the descriptor

Georgia Tech

Abstract



- DTCU is a state machine
  1. START
  2. DESC_REQ
  3. DESC_WAIT
  4. OP_REQ
  5. OP_WAIT
  6. EXECUTE
  7. OUT_REQ
  8. OUT_WAIT
- Stores operands / accumulator to internal buffers
  - Like Virgo
- Can perform MMA on GEMM bigger than the size of native tiles
  - Internally divides into native tile sizes and iterate
- Variable Tile Shape

Georgia Tech

Descriptor

```
class DTensorCore : public SimObject<DTensorCore> {
public:
  struct Desc {
    uint64_t ptrA;
    uint64_t ptrB;
    uint64_t ptrC;
    uint64_t ptrD;
    uint32_t ldmA;
    uint32_t ldmB;
    uint32_t ldmC;
    uint32_t ldmD;
    uint16_t M;
    uint16_t N;
    uint16_t K;
    uint8_t  fmt_s;
    uint8_t  fmt_d;
    uint8_t  flags;
    uint8_t  shape_n_size;
    uint16_t shape_policy;
    uint32_t reserved2;
  };
```

- Exactly 64B
- ptrA – D
  - Operand/Accumulator/Output memory address
- ldmA – D
  - Leading dimensions
- M/N/K
  - Total GEMM Size
- fmt_s / d
  - Source / Destination(accumulator) element type
- flags
  - 0 = Set accumulator = ptrC
  - 1 = Set accumulator = 0
- shape_n_size
  - Manually selects N-dimension
- shape_policy
  - 0 = Manual
  - 1 = Automatically chooses N-dimension (TBD)

Georgia Tech

Tile Shape

- Tile shapes of Virgo and Hopper's warp-group MMA
  - Virgo - Fixed at 128 × 64 × 128
  - Hopper - M = 64, N is selected by software, K depends on input data type
- M-dimension is fixed at 64
- N-dimension is selected by software using descriptor
  - Host can select N-dimension from the legal set (in multiples of 16 up to 128)
    - i.e. shape_n_size = 1-8 (=16, 32, …, 128)
- K-dimension depends on input data type
  - fp16 → 16
  - fp32 → 8
- Partial tiles are not supported
  - M/N/K must be exact multiples of native tile size
  - Falls back during initialization

Cache Simulator on SimX

- SimX has a cache timing model in *cache_sim.cpp*
- Caches (cache_sim.cpp) does not actually transfer data
  - For modelling transaction history
    - R/W counts
    - Cache hit/miss count
    - # of delayed cycles
- *mem_req_out* / *mem_rsp_in* simulates cache-side request / response timing
- Actual data is directly sourced from memory
- DTCU calculates the number of cache lines needed for operands / output
- Sequentially requests *mem_req_out* and wait until *mem_rsp_in*
  - Simulates the data transaction via L2
- Then, reads actual data off the RAM

Initialization (START / DESC_REQ / DESC_WAIT)

```
tile_m_ = 64; // fixed tile M dimension
tile_n_ = uint32_t(desc_.shape_n_size) * 16; // tile N dim
tile_k_ = 8 * (4 / in_sz); // tile K dimension is determin

// Initialize internal buffers based on tile sizes
a_buf_.assign(tile_m_ * 8, 0);
b_buf_.assign(8 * tile_n_, 0);
accum_buf_.assign(tile_m_ * tile_n_, 0.0f);

// Calculate # of tiles required to cover the entire GEMM
tiles_m_ = desc_.M / tile_m_;
tiles_n_ = desc_.N / tile_n_;
tiles_k_ = desc_.K / tile_k_;

// Initialize tile indices to start from the first tile
tile_m_idx_ = 0;
tile_n_idx_ = 0;
tile_k_idx_ = 0;
total_op_reqs_ = 0;
total_out_reqs_ = 0;
```

- D_TENSOR_START starts the DTCU
  - Sets *busy_* as true
  - Moves to DESC_REQ
- DESC_REQ
  - Issues one L2 request for descriptor
    - Descriptor is 64B = 1 cache line = 1 request
  - Moves to DESC_WAIT
- DESC_WAIT
  - Waits for the descriptor response
  - Loads descriptor from the RAM
  - Prepares tile-related states
    - Sets the native tile shape
    - Computes total # of tiles needed
      - tiles_m_, tiles_n_, tiles_k_
    - Initialize operands / accumulator buffers
  - Builds cache-line request lists for the first tile
  - Move to OP_REQ

Load Operands (OP_WAIT / OP_REQ)

- Loads operands for the current tile
  - Determined by *tile_k_idx, tile_n_idx, tile_m_idx*
- OP_REQ
  - Issues one L2 request at a time
    - Requests are generated one per touched cache line
  - Moves to OP_WAIT after each request
  - If no requests remain, clears the pending tag
- OP_WAIT
  - Waits for the matching response
  - If more requests remain, returns to OP_REQ and issue next read
  - When all requests completed calls *load_operands()*
    - Reads operand data from RAM into internal buffers
    - Initialize accumulator buffer if it's the first tile
  - Estimates EXECUTE latency by using the same model used for in-core WMMA micro-operations
  - 4 cycles per equivalent micro-op
- Moves to EXECUTE

Execute MMA (EXECUTE)

- First waits for the estimated execution latency
  - Micro-operation count is used only for timing, not for numeric computation
- Computes the current K-tile for the current output tile
  - Current tile is determined by *tile_k_idx*, *tile_n_idx*, *tile_m_idx*
- Uses preloaded buffers and calls *execute_mma()*
- Reuses the same FMA / FEDP arithmetic path used from in-core TCU
- Iterates over all *(m, n)* elements
  - Packs A-row, B-column and accumulator into array that FEDP takes in
  - Performs dot-product accumulation across the current K-tile
    - fedp(a_words, b_words, acc_bit)
    - FEDP internally applies multiple FMA operations
- Updates accumulator buffer in place
- If more K-tiles remain, return to OP_REQ
  - Increase *tile_k_idx*
  - Rebuild cache line request list
- If this is the last K-tile, move to OUT_REQ

Store Output (OUT_REQ / OUT_WAIT)

- OUT_REQ
  - Issues one L2 write request for the current output cache line
    - Requests are generated one per touched cache line
  - Moves to OUT_WAIT
- OUT_WAIT
  - Waits until the corresponding L2 write response arrives
  - If more cache lines remain, returns to OUT_REQ and issues the next write
  - When all requests completed calls *store_output()*
    - Writes *accum_buf_* to memory as the final D tile
    - If more output tiles remain in M/N, rebulid request lists and go back to OP_REQ
      - Re-calculate # of cache line needed and move to OP_REQ
    - Otherwise, clear *busy_,* and set *done_*

DTCU vs. In-core TCU

DTCU

- Performs MMA asynchronous from SIMT core
- Descriptor-based
  - Kernel only issues *dtensor_start,* and waits with *dtensor_poll()*
- Internally iterates over K-tiles and output tiles by state machines
- Uses internal buffers
  - Operands and partial sums in buffers *(a_buf_, b_buf_, accum_buf_)*
- Larger granularity
  - Native tile of 64 x (16~128) x (data type)
- Issues L2 requests and write final output back itself

In-core TCU

- MMA operation executed on SIMT core
- Kernel-driven
  - Kernel issues *load_matrix_sync*, *mma_sync* and *store_matrix_sync*
- Kernel iterates over K-tiles and output tiles explicitly
- Uses fragments
  - Operands and partial sums in warp-level fragments *(fragA, fragB, fragD)*
- Smaller granularity
  - Defined by *wmma_config_t* and per-thread fragment mapping
- Uses normal core LD/ST for operand movement

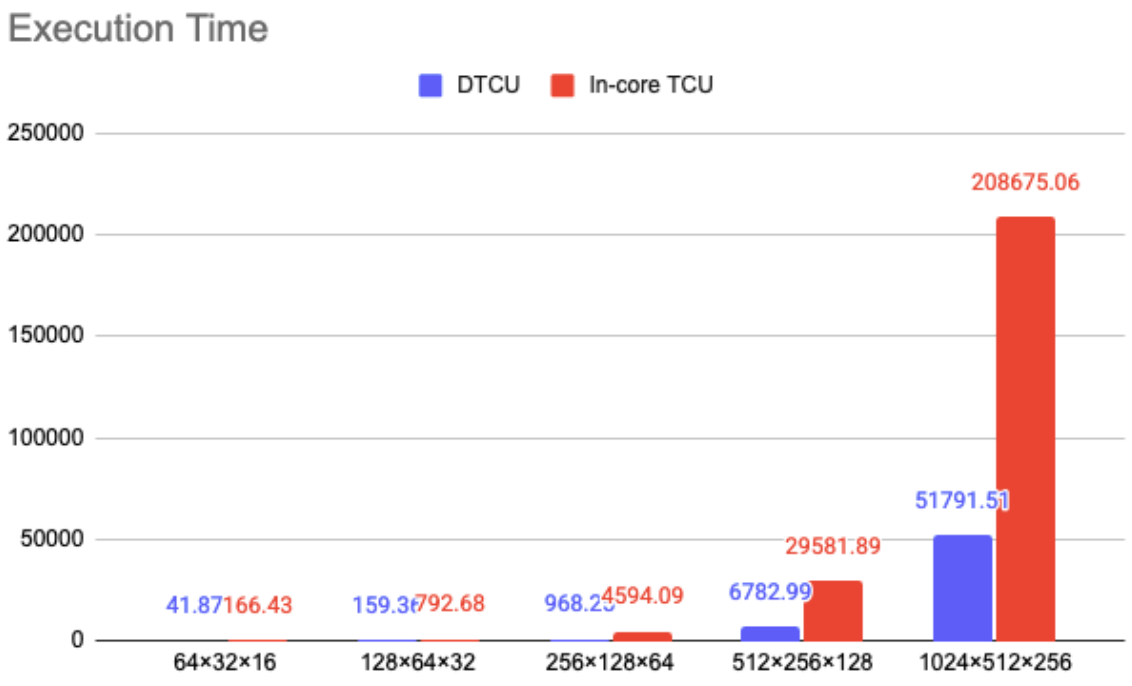
Test

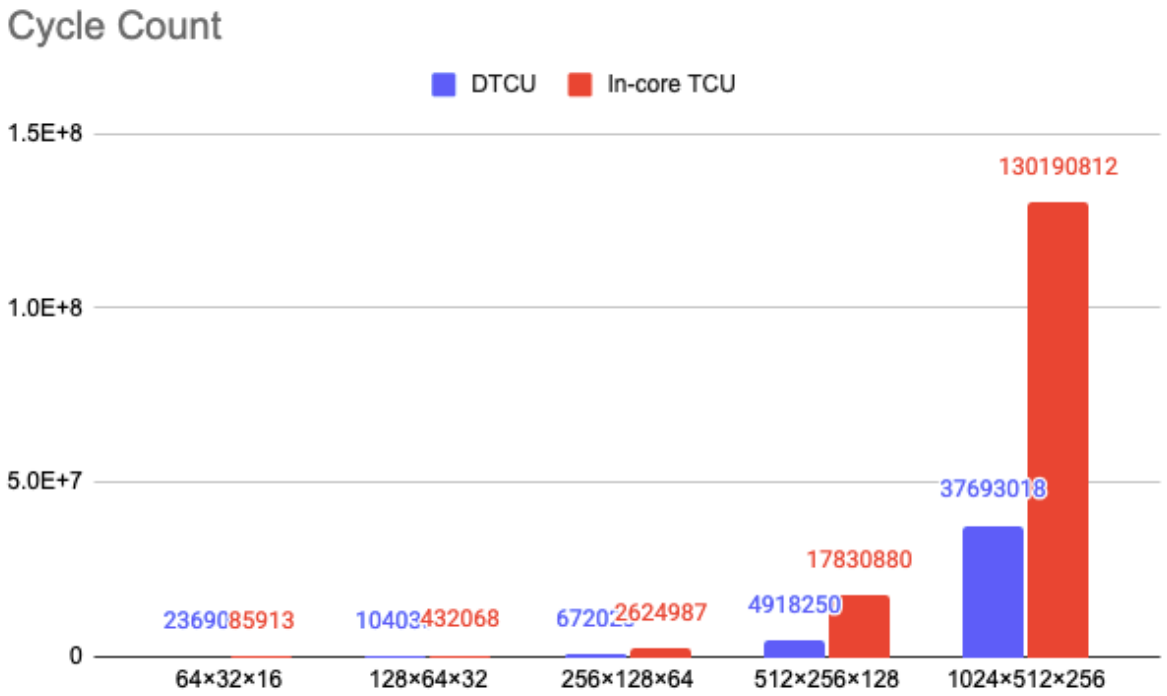
```
dtcu_compare: ---------------------- Running In-core TCU ----------------------
dtcu_compare: tcu - open device connection
dtcu_compare: tcu - upload program
dtcu_compare: tcu - download destination buffer
PERF: core0: lmem: reqs=0, bank_stalls=0 (utility=0%)
PERF: core0: coalescer: misses=43240
PERF: core0: icache: reads=64764, miss=51 (hit=100%), mshr_st=0 (utility=100%)
PERF: core0: dcache: reqs=45383, miss_r=2403 (hit=93%), miss_w=5660 (hit=46%), bank_
st=0 (utility=100%)
PERF: core0: l2cache: reqs=12169, miss_r=767 (hit=55%), miss_w=4300 (hit=59%)
PERF: memory: reqs=11276 (r=770, w=10506), lat=29.9 cyc, bank_st=0 (utility=100%)
PERF: instrs=64757, cycles=432068, IPC=0.150
dtcu_compare: ---------------------- Running DTCU ----------------------
dtcu_compare: dtcu - open device connection
dtcu_compare: dtcu- upload program
[DTCU] ptrA=0x10000 ptrB=0x12000 ptrC=0x13000 ptrD=0x1b000 ldmA=32 ldmB=32 ldmC=64 l
dmD=64 M=128 N=64 K=32 fmt_s=1 fmt_d=0 flags=0 shape_n_size=2 shape_policy=0 tileM=6
4 tileN=32 tileK=16
[DTCU] L2 MemReq count: desc=1, op=1280, output=512, total=1793
dtcu_compare: dtcu - download destination buffer
PERF: core0: lmem: reqs=0, bank_stalls=0 (utility=0%)
PERF: core0: coalescer: misses=264
PERF: core0: icache: reads=9226, miss=44 (hit=100%), mshr_st=0 (utility=100%)
PERF: core0: dcache: reqs=362, miss_r=17 (hit=84%), miss_w=219 (hit=14%), bank_st=0
(utility=100%)
PERF: core0: l2cache: reqs=2117, miss_r=760 (hit=43%), miss_w=713 (hit=8%)
PERF: memory: reqs=1565 (r=763, w=802), lat=23.3 cyc, bank_st=0 (utility=100%)
PERF: instrs=9219, cycles=104038, IPC=0.089
dtcu_compare: ---------------------- RESULT ----------------------
M=128 N=64 K=32

[In-core TCU]
  host_ms=792.681 cycles=432068 instrs=64757 IPC=0.150
  loads=0 stores=0 stall_lsu=767 stall_tcu=0 instr_lsu=0 instr_tcu=0
  l2_reads=1687 l2_writes=10482 mem_reads=770 mem_writes=10506

[DTCU]
  host_ms=159.363 cycles=104038 instrs=9219 IPC=0.089
  loads=0 stores=0 stall_lsu=760 stall_tcu=0 instr_lsu=0 instr_tcu=0
  l2_reads=1339 l2_writes=778 mem_reads=763 mem_writes=802

[Ratio DTCU / TCU]
  host_ms=0.201 cycles=0.241 l2_total=0.174 mem_total=0.139
PASSED
```

- Runs large GEMM over multiple tiles
  - First runs in-core TCU
    - 1 core, 1 warp, 4 threads
    - Equivalent tile size at (8×4×8)
  - Then runs DTCU
    - Tile size at (64×32×16)
  - Verified correctness of outputs against CPU and each other
- Compared performance metrics across 5 GEMM with different sizes
  - (64×32×16) / (128×64×32) / (256×128×64) / (512×256×128) / (1024×512×256)
  - Measured execution time between *vx_start* and *vx_ready_wait*
    - Captures kernel execution + DTCU completion latency
    - Excludes memory allocation, host-side operands & descriptors setup

Georgia Tech

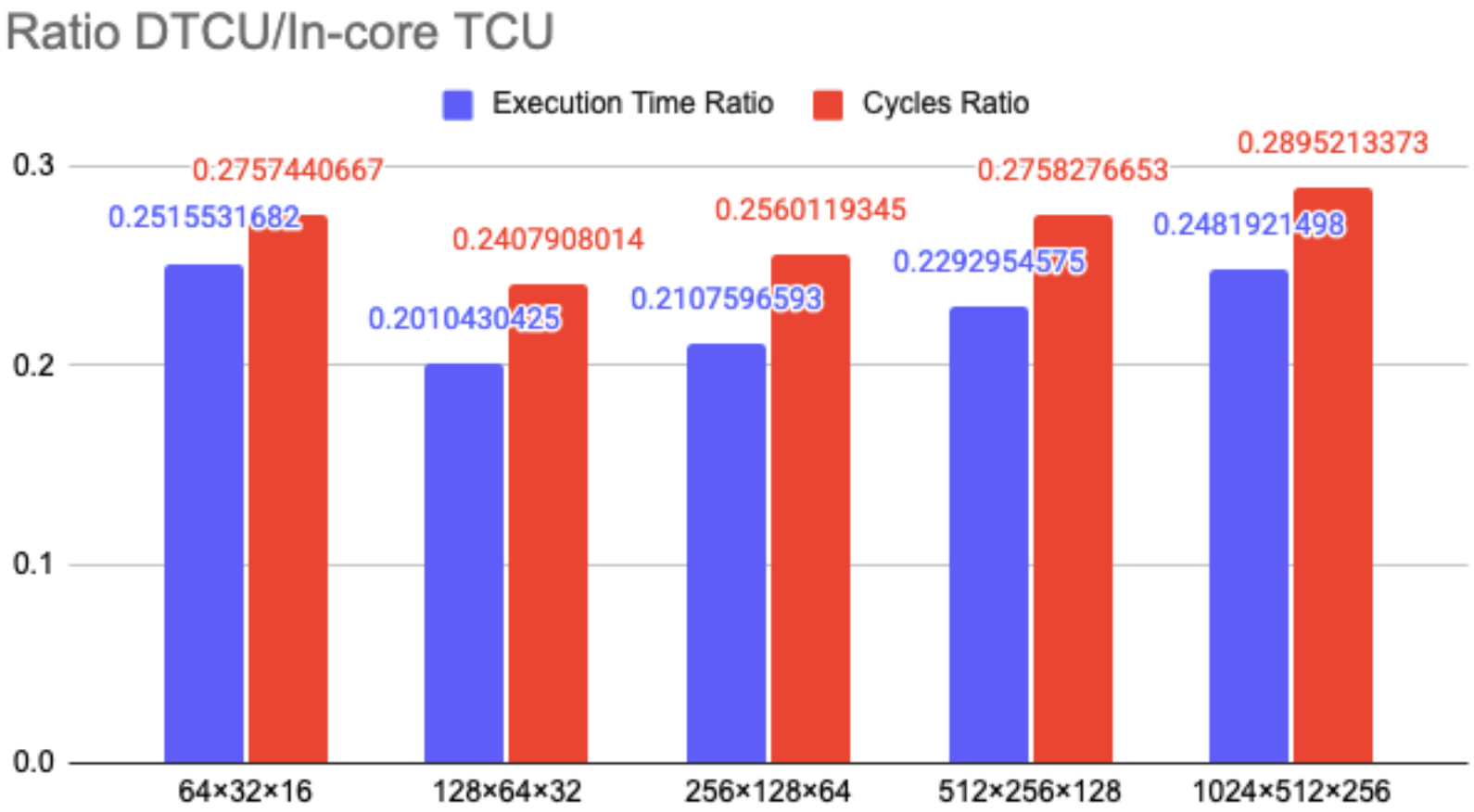
Performance Comparison





Georgia Tech

Performance Comparison



Georgia Tech